των τιμών που εισήχθησαν. Τα αποτελέσματα που παράγει η συνάρτηση πέραν της τιμής επιστροφής της είναι γνωστά ως παράπλευρα αποτελέσματα (side effects). Η Java, όπως θα δούμε αμέσως παρακάτω σε αυτήν την ενότητα, υποστηρίζει συναρτήσεις που παράγουν μόνο παράπλευρα αποτελέσματα χωρίς να επιστρέφουν κάποια τιμή. Άλλες γλώσσες διαφοροποιούν τα υποπρογράμματα που παράγουν μόνο παράπλευρα αποτελέσματα από αυτά που επιστρέφουν κάποια τιμή. Για παράδειγμα, η Pascal ονομάζει τα υποπρογράμματα που δεν επιστρέφουν κάποια τιμή διαδικασίες (procedures), ενώ διατηρεί την ονομασία συναρτήσεις (functions) για τα υποπρογράμματα που επιστρέφουν τιμή.

Επάνω στη δυνατότητα να αναπτύσσουμε συναρτήσεις βασίζεται η δυνατότητα να οργανώνουμε τους πηγαίους κώδικες με δομημένο τρόπο. Δημιουργείται έτσι το μοντέλο του δομημένου προγραμματισμού (structured programming). Πρόκειται για ένα σημαντικό μοντέλο προγραμματισμού που διευκολύνει την ανάπτυξη και συντήρηση των προγραμμάτων, ενώ αποτελεί συστατικό στοιχείο του Αντικειμενοστρεφούς μοντέλου.

Η Java υποστηρίζει δύο ειδών συναρτήσεις, τις στατικές (static) και τις μη στατικές (non static). Το κύριο θέμα αυτού του κεφαλαίου είναι οι στατικές συναρτήσεις. Οι μη στατικές συναρτήσεις και μεταβλητές παρουσιάζονται αργότερα σε αυτό το εγχειρίδιο κατά την εισαγωγή στο Αντικειμενοστρεφές μοντέλο, στην ενότητα 11. Εδώ μαζί με την παρουσίαση των στατικών συναρτήσεων, παρουσιάζουμε και τις στατικές μεταβλητές της κλάσης.

7.1 Στατικές Συναρτήσεις

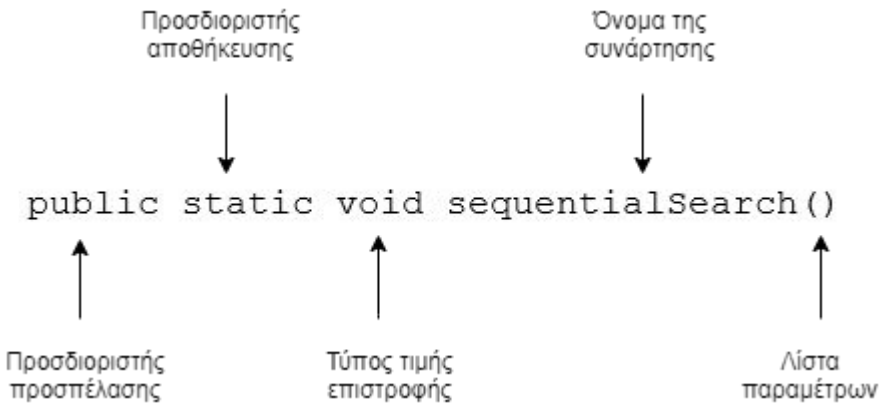
Έχουμε ήδη χρησιμοποιήσει στατικές συναρτήσεις στην πολύ απλή τους μορφή. Ας εξετάσουμε καταρχάς τη συνάρτηση sequentialSearch του κώδικα 6.12. Το τμήμα

```
public static void sequentialSearch()
```

ονομάζεται επικεφαλίδα της συνάρτησης (function header). Το μπλοκ που ακολουθεί την επικεφαλίδα ονομάζεται σώμα της συνάρτησης (function body). Η επικεφαλίδα της συνάρτησης ονομάζεται και διεπαφή της συνάρτησης (function interface), γιατί είναι το τμήμα τις λεπτομέρειες του οποίου είναι αναγκαίο να γνωρίζει ο χρήστης της συνάρτησης για να είναι σε θέση να την καλεί και να λαμβάνει την επιστροφή της συνάρτησης. Η επικεφαλίδα μαζί με το σώμα της συνάρτησης αποτελούν τον ορισμό της συνάρτησης (function definition). Ο ορισμός της συνάρτησης από μόνος του δεν παράγει κάποιο αποτέλεσμα σε ένα πρόγραμμα. Το αποτέλεσμα το παράγει η κλίση της συνάρτησης. Για παράδειγμα, η sequentialSearch χρειάζεται μια ακέραια μεταβλητή που ονομάζει schElement. Μνήμη για αυτήν τη μεταβλητή δεσμεύεται μόνον κατά την κλίση της συνάρτησης.

Στην αρχή της επικεφαλίδας έχουμε έναν προσδιοριστή προσπέλασης, στη συγκεκριμένη περίπτωση τον προσδιοριστή public. Για τους προσδιοριστές προσπέλασης έχουμε ήδη μιλήσει στην ενότητα 2.3. Στη συνέχεια ακολουθεί ο προσδιοριστής αποθήκευσης (class storage specifier), static που καθορίζει πως πρόκειται για στατική συνάρτηση. Μία στατική συνάρτηση μπορεί να κληθεί μέσω της κλάσης στην οποία ορίζεται όπως φαίνεται στον κώδικα 2.6.

Στη συνέχεια ακολουθεί η δεσμευμένη λέξη void που ορίζει πως η συνάρτηση αυτή δεν έχει τιμή επιστροφής. Μετά ακολουθεί το όνομα της συνάρτησης που είναι sequentialSearch και τέλος πριν το σώμα της συνάρτησης παρεμβάλλεται η λίστα παραμέτρων που στο παράδειγμά μας είναι κενή. Στο σχήμα 7.1 παρουσιάζονται τα συστατικά μέρη της διεπαφής της συνάρτησης.



Σχήμα 7.1 *Η διεπαφή της συνάρτησης*

Συνεχίζοντας την εξέταση της sequentialSearch παρατηρούμε πως αυτή ψάχνει να εντοπίσει μια συγκεκριμένη τιμή σε έναν συγκεκριμένο πίνακα. Μάλιστα τιμή αναζήτησης και πίνακας ορίζονται μέσα στην ίδια τη συνάρτηση. Αυτό σημαίνει πως η συνάρτηση αυτή δεν μπορεί να χρησιμοποιηθεί για κανέναν άλλο πίνακα αλλά ούτε και για να ψάξει άλλη τιμή έστω και στον ίδιο πίνακα. Ο αλγόριθμος όμως της σειριακής αναζήτησης είναι ίδιος για οποιονδήποτε μονοδιάστατο πίνακα και για οποιαδήποτε τιμή αναζήτησης. Επομένως, θα ήταν χρήσιμο να απομονώσουμε τον αλγόριθμο της σειριακής αναζήτησης κατά τρόπο ώστε να μπορούμε να τον εφαρμόζουμε σε οποιονδήποτε πίνακα και για οποιαδήποτε τιμή. Λύση σε αυτό το ζήτημα παρέχει η λίστα παραμέτρων της συνάρτησης. Η λίστα παραμέτρων δίνει τη δυνατότητα σε αυτόν που καλεί τη συνάρτηση να καθορίζει τόσο τον πίνακα όσο και την τιμή στα οποία επενεργεί η συνάρτηση. Όμως για να επιτευχθεί αυτό θα πρέπει κατά τον ορισμό της συνάρτησης να καθορίσουμε τους τύπους των παραμέτρων, όπως δείχνει παρακάτω η νέα διεπαφή της συνάρτησης.

```
public static void sequentialSearch(int[] array, int
schElement)
```

Η νέα διεπαφή επιβάλλει ώστε κατά την κλήση της συνάρτησης να δίνεται ένας πίνακας ακέραιων και μια ακέραια τιμή. Είναι προφανές πως κάποιες μεταβολές θα πρέπει να γίνουν στο σώμα της sequentialSearch. Ας δούμε όμως πρώτα πώς καλείται μια συνάρτηση με τη νέα διεπαφή της sequentialSearch.

```
int[] t1 = {3, 2, 5, 3, 6, 9};
int[] t2 = {6, 4, 5, 2, 1, 9, 9};
int sItem = 5;
sequentialSearch(t1, 9);
sequentialSearch(t2, sItem);
```

Κώδικας 7.1 *Κλήση της sequentialSearch με παραμέτρους, χωρίς τιμή επιστροφής*

Στον κώδικα 7.1 βλέπουμε δύο κλήσεις της sequentialSearch. Κατά την πρώτη κλήση η συνάρτηση αναζητά στον πίνακα t1 την τιμή 9 και στη δεύτερη κλήση αναζητά στον πίνακα t2 την τιμή της μεταβλητής sItem, δηλαδή την τιμή 5.

Ας σημειωθεί πως οι παράμετροι στον ορισμό της συνάρτησης ονομάζονται τυπικές παράμετροι (formal parameters), ενώ οι παράμετροι που δίνει ο χρήστης της συνάρτησης κατά την κλήση της ονομάζονται πραγματικές παράμετροι (actual parameters).

Το πρόβλημα στον σχεδιασμό της συνάρτησης όπως τον έχουμε τώρα είναι πως η συνάρτηση δεν επιστρέφει στον κώδικα που την καλεί το αποτέλεσμα της αναζήτησης. Αυτό όμως είναι αναγκαίο καθώς στο αποτέλεσμα της αναζήτησης, ο κώδικας που την καλεί θα βασίσει την περαιτέρω ροή του. Για παράδειγμα, ενδέχεται αν η τιμή βρεθεί, ο χρήστης της συνάρτησης να επιθυμεί να αυξήσει την τιμή στον πίνακα κατά 1. Αν πάλι η τιμή δεν βρεθεί, μπορεί να μην χρειάζεται καμία περαιτέρω ενέργεια για τον συγκεκριμένο πίνακα.

Το πρόβλημα αυτό λύνεται με την τιμή επιστροφής της συνάρτησης. Στο συγκεκριμένο παράδειγμα μια καλή τιμή επιστροφής είναι τύπου int. Στην περίπτωση που θα βρεθεί η τιμή αναζήτησης, ο ακέραιος που θα επιστραφεί θα αναπαριστά τη θέση της τιμής αναζήτησης στον πίνακα. Αν πάλι η τιμή δεν βρεθεί, μπορεί να επιστραφεί η τιμή -1. Καθώς δεν υπάρχει θέση του πίνακα με αρνητική τιμή, η τιμή -1 μπορεί να αναπαραστήσει

με ασφάλεια την αποτυχία της αναζήτησης. Προσέξτε εδώ πως αν διαλέξουμε να αναπαραστήσουμε την αποτυχία αναζήτησης με μη αρνητικό αριθμό, π.χ. με 0, τότε έχουμε κακό σχεδιασμό της συνάρτησης. Αυτό γιατί η συνάρτηση θα επιστρέφει 0, τόσο στην περίπτωση που το στοιχείο αναζήτησης δεν βρεθεί, όσο και στην περίπτωση που βρεθεί στη θέση 0 του πίνακα.

Επομένως, η διεπαφή της συνάρτησης τροποποιείται όπως δείχνει ο ακόλουθος κώδικας:

```
public static int sequentialSearch(int[] array, int schElement)
```

Η τιμή επιστροφής έγινε int από void που ήταν στην προηγούμενη έκδοση. Τώρα η συνάρτηση μπορεί να κληθεί όπως δείχνει ο κώδικας 7.2:

```
static void callSeqSearch() { //Κώδικας 7.2
    int[] t1 = {3, 2, 5, 3, 6, 9};
    int sItem = 5;
    int idx = sequentialSearch(t1, sItem);
    if (idx > -1) {
        t1[idx]++;
    }
}
```

Κώδικας 7.2 *Κλήση της sequentialSearch με παραμέτρους και τιμή επιστροφής*

Στον κώδικα 7.2, η κλήση της συνάρτησης επιστρέφει τη θέση του πίνακα στην οποία βρέθηκε το sItem ή -1. Αν επιστρέψει -1, καμία ενέργεια επάνω στον πίνακα δεν γίνεται. Αν όμως επιστρέψει μη αρνητική τιμή, τότε προσαυξάνεται η τιμή στη θέση που επιστράφηκε κατά 1.

Ακόμη και αν κάνουμε τις αλλαγές που συζητήσαμε μέχρι τώρα, παραμένει ένα σοβαρό μειονέκτημα στη συνάρτηση. Αυτό συνίσταται στο γεγονός ότι η συνάρτηση εμφανίζει τα αποτελέσματα της αναζήτησης. Προσέξτε, το μειονέκτημα δεν είναι ότι επιστρέφει στον κώδικα που την καλεί τα αποτελέσματα της αναζήτησης αλλά ότι τα εμφανίζει χρησιμοποιώντας μάλιστα την println. Αυτό το μειονέκτημα πολλές φορές γίνεται δύσκολα κατανοητό στους αρχάριους προγραμματιστές. Πράγματι, αν κάνω έναν κώδικα που ψάχνει σε συγκεκριμένο πίνακα για να βρει μια συγκεκριμένη τιμή και το μόνο που θέλω είναι να μάθω αν η τιμή βρέθηκε και που, τότε δεν είναι πρόβλημα αν ο κώδικας εμφανίζει το αποτέλεσμα της αναζήτησης. Μια συνάρτηση όμως μπορεί να κληθεί σε ένα σημείο που θέλω να εμφανίσω το αποτέλεσμά της αλλά μπορεί να κληθεί και σε πολλά σημεία που δεν θέλω να το εμφανίζω αλλά απλώς το χρειάζομαι μέσα στον κώδικά μου. Για παράδειγμα, μπορεί την sequentialSearch να την καλέσω σε μια παραθυρική εφαρμογή. Σε αυτήν την περίπτωση η έξοδος με την println που είναι κατάλληλη μόνο για περιβάλλοντα γραμμής εντολών (command line) προφανώς θα μου καταστρέψει τη διεπαφή χρήστη (user interface).

Ο κώδικας 7.3 δίνει τον ορισμό της sequentialSearch που ενσωματώνει όλες τις παρατηρήσεις μέχρι εδώ.

```
public static int sequentialSearch(int[] array, int sItem) {
    for (int i = 0; i < array.length; i++) {
        if (array[i] == sItem) {
            return i;
        }
    }
    return -1;
}
```

Κώδικας 7.3 *Συνάρτηση σειριακής αναζήτησης σε μονοδιάστατο πίνακα ακέραιων*

Η sequentialSearch σε αυτή τη μορφή μπορεί να κληθεί από οποιονδήποτε κώδικα για να αναζητήσει σε οποιονδήποτε μονοδιάστατο πίνακα ακέραιων οποιαδήποτε ακέραια τιμή. Προσέξτε πως η επιστροφή τιμής από τη συνάρτηση γίνεται με χρήση της δεσμευμένης λέξης return. Η λέξη return συντάσσεται με έκφραση που ο τύπος της είναι υποχρεωτικά ίδιος ή συμβατός με τον τύπο της συνάρτησης. Η return προκαλεί άμεση έξοδο από τη συνάρτηση. Σε συναρτήσεις τύπου void, η return συντάσσεται μόνη, ακολουθούμενη απλώς από τον χαρακτήρα ελληνικό ερωτηματικό (semicolon). Σε void συναρτήσεις, η return δεν είναι υποχρεωτική. Ωστόσο, σε κάποιες περιπτώσεις είναι χρήσιμη.

Οι παράμετροι των συναρτήσεων στην Java είναι παράμετροι θέσης (positional parameters). Αυτό σημαίνει πως κατά την κλήση συνάρτησης, οι πραγματικές παράμετροι αντιστοιχίζονται με τις τυπικές παραμέτρους ανάλογα με τη θέση που διατηρεί κάθε μία στην λίστα παραμέτρων.

Τέλος να επισημάνουμε πως οι συναρτήσεις στα πλαίσια του αντικειμενοστρεφούς προγραμματισμού ονομάζονται και μέθοδοι (methods).

7.2 Υπερφόρτωση Συναρτήσεων

Πάντως, είναι πιθανό να χρειάζεται να κάνουμε αναζήτηση τιμών και σε πίνακες που δεν είναι πίνακες ακέραιων. Ας υποθέσουμε πως χρειάζεται να κάνουμε αναζήτηση σε έναν πίνακα πραγματικών. Σε αυτήν την περίπτωση μπορούμε να υλοποιήσουμε μια άλλη συνάρτηση με παραμέτρους τύπου double[] και double. Το θέμα είναι πως παρότι αλλάζουν οι τύποι των δεδομένων, η λειτουργία παραμένει ίδια. Δεν θέλουμε για την ίδια λειτουργία να έχουμε συναρτήσεις με διαφορετικό όνομα. Παρόλα αυτά, για κάποιες γλώσσες προγραμματισμού, η μόνη λύση είναι να κάνουμε μια συνάρτηση που έχει διαφορετικό όνομα από τη συνάρτηση που ψάχνει σε πίνακες ακέραιων. Αυτό όμως δεν είναι βολικό γιατί δεν θέλουμε να αναγκαζόμαστε να εφευρίσκουμε και να θυμόμαστε διαφορετικά ονόματα για την ίδια λειτουργία. Ευτυχώς, η Java όπως και οι περισσότερες σύγχρονες γλώσσες προγραμματισμού, επιλύουν αυτό το θέμα με την υπερφόρτωση συναρτήσεων (function overloading).

Με την υπερφόρτωση συναρτήσεων μπορούμε να ορίσουμε συναρτήσεις με το ίδιο όνομα αρκεί να διαφέρει η ταυτότητά τους (function signature). Η ταυτότητα μιας συνάρτησης αποτελείται από το όνομα της συνάρτησης και τη λίστα παραμέτρων. Προσέξτε πως η τιμή επιστροφής δεν συμπεριλαμβάνεται στην ταυτότητα. Επομένως, μπορούμε να κάνουμε συναρτήσεις με το ίδιο όνομα, αρκεί να διαφέρει η λίστα παραμέτρων. Ποια από δύο συναρτήσεις που έχουν το ίδιο όνομα θα κληθεί είναι μια απόφαση που λαμβάνει ο μεταγλωττιστής ανάλογα με τις πραγματικές παραμέτρους. Στον κώδικα 7.4 δίνουμε παράδειγμα ορισμού και χρήσης υπερφορτωμένων συναρτήσεων.

```
static void f(int i) { //Κώδικας 7.4
      System.out.println("f(int)");
  }

  static void f(double d) { //Κώδικας 7.4
      System.out.println("f(double)");
  }

  public static void main(String[] args) { //Κώδικας 7.4
      f(1);
      f(1d);
  }
```

Κώδικας 7.4 *Ορισμός και χρήση υπερφορτωμένων συναρτήσεων*

Στον κώδικα 7.4 και οι δύο συναρτήσεις που ορίζουμε ονομάζονται f. Η μία όμως δέχεται μια ακέραια παράμετρο και η άλλη μια πραγματική παράμετρο. Στην main καλούμε την f την πρώτη φορά με ακέραια παράμετρο και τη δεύτερη με πραγματική. Η έξοδος του κώδικα είναι:

```
f(int)
f(double)
```

Η έξοδος αυτή μας δείχνει πως την πρώτη φορά κλήθηκε η f που λαμβάνει την ακέραια παράμετρο και τη δεύτερη φορά κλήθηκε η f που λαμβάνει την πραγματική παράμετρο.
Έτσι, αν θέλουμε να έχουμε και μια sequentialSearch ώστε να ψάχνει και σε πραγματικούς πίνακες δεν έχουμε παρά να την υπερφορτώσουμε όπως δείχνει ο κώδικας 7.5.

```
public static int sequentialSearch(double[] array, double sItem) {
    for (int i = 0; i < array.length; i++) {
        if (approximateEquals(array[i], sItem, 0.0001)) {
            return i;
```

```
            }
        }
        return -1;
    }
```

Κώδικας 7.5 *Σειριακή αναζήτηση σε πίνακες πραγματικών*

7.3 Παράμετροι

Έχουμε ήδη συζητήσει σχετικά με τις μεταβλητές αναφοράς και τις μεταβλητές τιμής. Η συμπεριφορά αυτών των δύο τύπων μεταβλητών είναι διαφορετική όταν περνάνε σαν παράμετροι σε συναρτήσεις. Σε αυτήν την ενότητα εξετάζουμε πρώτα τις μεταβλητές τιμής ως παραμέτρους και στη συνέχεια τις μεταβλητές αναφοράς.

7.3.1 Οι μεταβλητές τιμής ως παράμετροι

Ο κώδικας 7.6 παρουσιάζει δύο συναρτήσεις, την increment και την main. Μελετήστε τον κώδικα και επιχειρήστε να υποθέσετε την έξοδό του.

```
static void increment(int i) {
    i++;
    System.out.println(i);
}

public static void main(String[] args) {
    int i = 0;
    increment(i);
    System.out.println(i);
}
```

Κώδικας 7.6 *Παράδειγμα παραμέτρων τιμής*

Η συνάρτηση increment λαμβάνει ως παράμετρο έναν ακέραιο. Το αναγνωριστικό της παραμέτρου είναι i. Στην πρώτη γραμμή αυξάνει την τιμή της παραμέτρου κατά 1 και στη συνέχεια την τυπώνει.

Η main ορίζει μια ακέραια μεταβλητή, επίσης με το αναγνωριστικό i και την αρχικοποιεί στο 0. Στη συνέχεια καλεί την increment με πραγματική παράμετρο το i. Μετά το πέρας της increment, η main τυπώνει την i.

Πρώτη σημαντική παρατήρηση είναι ότι η i της increment και η i της main δεν ταυτίζονται. Μπορεί να είναι και οι δύο τύπου int και να έχουν το ίδιο αναγνωριστικό, ωστόσο πρόκειται για διαφορετικές μεταβλητές. Εξάλλου θα μπορούσαν να έχουν διαφορετικά αναγνωριστικά. Αν μεταβάλλετε το αναγνωριστικό στην main, π.χ. το κάνετε j, θα διαπιστώσετε πως δεν θα έχετε καμία διαφορά στο αποτέλεσμα του κώδικα.

Αν τρέξουμε τον κώδικα, θα διαπιστώσουμε πως η μεν increment τυπώνει 1, η δε main τυπώνει 0. Αυτή η συμπεριφορά εξηγείται ως εξής: Η main δεσμεύει χώρο στη μνήμη κατάλληλο για τη μεταβλητή i, δηλαδή χώρο κατάλληλο για έναν ακέραιο. Σε αυτήν τη θέση μνήμης εκχωρείται η τιμή 0. Όταν καλείται η increment, ένα αντίγραφο της πραγματικής παραμέτρου περνάει στην τυπική παράμετρο. Στη συνέχεια η i της increment λειτουργεί σαν τοπική μεταβλητή. Η increment δηλαδή δεσμεύει επίσης χώρο για έναν ακέραιο στον οποίο αντιγράφεται η τιμή της πραγματικής παραμέτρου. Από εκεί και μετά, η increment επενεργεί επάνω στη δική της μεταβλητή i που είναι εντελώς ανεξάρτητη από την i της main. Με το τέλος εκτέλεσης της increment, ο έλεγχος επανέρχεται στην main. Η μεταβλητή i της main έχει διατηρήσει την τιμή της και έτσι εμφανίζεται το 0.

Οι μεταβλητές όλων των θεμελιωδών τύπων στην Java είναι μεταβλητές τιμής και περνούν στις συναρτήσεις σαν παράμετροι τιμής.

7.3.2 Οι μεταβλητές αναφοράς ως παράμετροι

Είπαμε παραπάνω πως οι θεμελιώδεις τύποι περνάνε στις συναρτήσεις σαν παράμετροι τιμής. Το ίδιο ισχύει και για τις μεταβλητές αναφοράς. Ωστόσο η συμπεριφορά είναι διαφορετική. Μελετήστε τον κώδικα 7.7. Ποια είναι κατά την γνώμη σας η έξοδός του;

```
static void increment(int[] store) { //Κώδικας 7.7
    for (int i = 0; i < store.length; i++) {
        store[i]++;
    }
    System.out.println(Arrays.toString(store));
}

public static void main(String[] args) {
    int[] s = {1, 2, 3};
    increment(s);
    System.out.println(Arrays.toString(s));
}
```

Κώδικας 7.7 *Η μεταβλητή αναφοράς ως παράμετρος*

Η έξοδος του κώδικα 7.7 είναι

```
[2, 3, 4]
[2, 3, 4]
```

Αυτό σημαίνει πως όποιες αλλαγές έγιναν στον πίνακα μέσα στην increment ισχύουν και έξω από αυτήν. Η εξήγηση έχει ως εξής: Η μεταβλητή s είναι μία αναφορά που περιέχει τη διεύθυνση του πίνακα. Η αναφορά αυτή αντιγράφεται στην παράμετρο store της increment. Εφόσον η s είναι αντίγραφο της store, και οι δύο δείχνουν στην ίδια θέση μνήμης. Άρα η increment επενεργεί στον ίδιο πίνακα στον οποίο αναφέρεται η s. Όποια μεταβολή γίνει στα στοιχεία του πίνακα μέσω μιας αναφοράς σε αυτόν, ισχύει όταν τον προσπελάσουμε μέσω οποιασδήποτε άλλης αναφοράς.
Προσέξτε όμως μια άλλη επίπτωση. Μελετήστε τον κώδικα 7.8. Ποια είναι η έξοδός του κατά την άποψή σας;

```
static void assign(int[] store) { //Κώδικας 7.8
    int[] s = {4, 5, 6};
    store = s;
    System.out.println(Arrays.toString(store));
}

public static void main(String[] args) { //Κώδικας 7.8
    int[] table = {1, 2, 3};
    assign(table);
    System.out.println(Arrays.toString(table));
}
```

Κώδικας 7.8 *Η μεταβλητή αναφοράς ως παράμετρος τιμής*

Η έξοδος του κώδικα 7.8 έχει ως εξής:

```
[4, 5, 6]
[1, 2, 3]
```

Η έξοδος αυτή μας δείχνει πως μέσα στην assign, η store μεταβλήθηκε ώστε να δείχνει στον ίδιο πίνακα με την τοπική μεταβλητή s. Ωστόσο, η table συνεχίζει να αναφέρεται στον πίνακα που δημιουργήθηκε στην main. Στην πράξη, ένα αντίγραφο της table πέρασε στην store. Το αντίγραφο, δηλαδή η store, ενημερώθηκε ώστε να δείχνει σε άλλο πίνακα, όμως η table έμεινε ανέπαφη. Το ίδιο ακριβώς αποτέλεσμα θα είχαμε βέβαια αν ονομάζαμε store την table.

Συνοψίζοντας, σε σχέση με τις μεταβλητές αναφοράς ως παραμέτρους, ισχύει το εξής στην Java: Όταν μια μεταβλητή αναφοράς περνάει σαν παράμετρος σε συνάρτηση, η συνάρτηση μπορεί να μεταβάλει το περιεχόμενο της μνήμης στην οποία αναφέρεται η εν λόγω μεταβλητή, δεν μπορεί όμως να μεταβάλει το περιεχόμενο της ίδιας της μεταβλητής.

7.3.3 Λίστα παραμέτρων μεταβλητού μήκους

Ας υποθέσουμε τώρα πώς αναπτύσσουμε μια εφαρμογή στην οποία απαιτείται συχνά να υπολογίζουμε τον μέσο όρο μιας σειράς πραγματικών τιμών. Το θέμα όμως είναι πως σε ένα σημείο του κώδικα χρειάζεται να υπολογίσουμε τον μέσο όρο τεσσάρων πραγματικών τιμών, σε κάποιο άλλο σημείο χρειάζεται ο μέσος όρος επτά πραγματικών τιμών. Μία λύση θα ήταν να υλοποιήσουμε δύο συναρτήσεις. Η μία με 4 τυπικές παραμέτρους και η άλλη με επτά. Αν αργότερα χρειαστεί να υπολογίσουμε τον μέσο όρο πέντε πραγματικών τιμών, τότε θα πρέπει να προσθέσουμε τον ορισμό κατάλληλης συνάρτησης με πέντε τυπικές παραμέτρους. Η αντιμετώπιση αυτή όμως είναι προβληματική. Σε όλες τις περιπτώσεις, ο αλγόριθμος είναι ο ίδιος και το μόνο που αλλάζει είναι το πλήθος των δεδομένων.

Ευτυχώς για μας, η Java προσφέρει μια αποτελεσματική δυνατότητα, τη λίστα παραμέτρων μεταβλητού μήκους (variable length parameter list).

Ας δούμε στην πράξη πώς υλοποιείται και πώς αξιοποιείται μια συνάρτηση που υπολογίζει τον μέσο όρο μιας σειράς πραγματικών αριθμών.

```
    static double average(double... sequence) {
        double sum = 0;
        for (int i = 0; i < sequence.length; i++) {
            sum += sequence[i];
        }
        return sum / sequence.length;
    }

    public static void main(String[] args) {
        double d1 = 12, d2 = 13.5, d3 = 16.1;
        double av1 = average(d1, 14.5);
        double av2 = average(d1, d2, d3);
        double totalAv = average(av1, av2);
        DecimalFormat f = new DecimalFormat("###.##");
        System.out.println(f.format(av1) + " " + f.format(av2) + " " +
f.format(totalAv));
    }
```

Κώδικας 7.9 *Παράμετρος μεταβλητού μήκους*

Η συνάρτηση average στον κώδικα 7.9 περιέχει την παράμετρο μεταβλητού μήκους sequence. Οι τρεις τελείες που ακολουθούν τον τύπο double δηλώνουν πως πρόκειται για παράμετρο μεταβλητού μήκους. Μόνο μια παράμετρος μεταβλητού μήκους επιτρέπεται σε μία συνάρτηση. Αν μάλιστα η συνάρτηση έχει και άλλες παραμέτρους, τότε η παράμετρος μεταβλητού μήκους πρέπει να βρίσκεται τελευταία στη λίστα παραμέτρων.

Στην main, η average καλείται τρεις φορές. Την πρώτη με δύο πραγματικές παραμέτρους, μία μεταβλητή και μία σταθερά, τη δεύτερη με τρεις πραγματικές παραμέτρους και την τρίτη φορά καλείται πάλι με δύο πραγματικές παραμέτρους.

Μελετώντας την υλοποίηση της average μπορούμε να διαπιστώσουμε πως οι παράμετροι περνάνε στην πράξη σε έναν πίνακα τύπου double. Εκείνο δηλαδή που συμβαίνει είναι πως ο μεταγλωττιστής τοποθετεί τις τιμές των πραγματικών παραμέτρων σε κατάλληλο πίνακα και περνάει μια αναφορά σε αυτόν τον πίνακα στην average. Έτσι μέσα στη συνάρτηση έχουμε πρόσβαση στις τιμές των πραγματικών παραμέτρων μέσω της average την οποία διαχειριζόμαστε κανονικά ως τυπική αναφορά σε μονοδιάστατο πίνακα πραγματικών. Εξαιτίας αυτού του μηχανισμού, μπορούμε να καλέσουμε την average με πραγματική παράμετρο έναν πίνακα πραγματικών όπως δείχνει ο κώδικας που ακολουθεί:

```
double[] d={12,13.5,16.1};
double totalAv=average(d);
```

7.3.4 Οι παράμετροι της main

Μια εφαρμογή Java μπορεί να κληθεί από διάφορα περιβάλλοντα. Στην τυπική περίπτωση, οι εφαρμογές Java σε αυτό το βιβλίο καλούνται μέσα από το NetBeans. Ωστόσο, μια εφαρμογή Java μπορεί να κληθεί μέσα από

πολλά και διαφορετικά περιβάλλοντα. Για παράδειγμα, μπορεί να κληθεί από το powershell των Windows ή μέσα από μια άλλη εφαρμογή Java ή μέσα από οποιοδήποτε λειτουργικό στο οποίο τρέχει η JVM.

Επιπλέον, μια εφαρμογή θα πρέπει να έχει τρόπο ακριβώς όπως οι συναρτήσεις να λαμβάνει μια σειρά από παραμέτρους ώστε να διαφοροποιεί τη συμπεριφορά της ανάλογα με τις τιμές των παραμέτρων. Ωστόσο, τα περιβάλλοντα από τα οποία μπορεί να κληθεί μια εφαρμογή Java δεν υποστηρίζουν ακριβώς τους ίδιους τύπους δεδομένων που υποστηρίζει η Java. Προκύπτει επομένως η ανάγκη να υπάρχει μια διεπαφή ικανή να εκκινήσει μια εφαρμογή Java που να είναι ταυτοχρόνως συμβατή με τα διάφορα περιβάλλοντα κλήσης.

Την ανάγκη αυτή καλύπτει η συνάρτηση main. Η main αποτελεί την κύρια είσοδο σε κάθε εφαρμογή Java. Αυτό σημαίνει πως από οποιοδήποτε περιβάλλον και αν καλέσουμε μια εφαρμογή Java, η εικονική μηχανή θα ψάξει να βρει και να καλέσει την main. Στη συνέχεια, η main μπορεί να καλέσει ανάλογα με τον σχεδιασμό της εφαρμογής οποιαδήποτε συνάρτηση είναι στη διάθεση της εφαρμογής. Επομένως, υπάρχει η ανάγκη να παρέχεται ένας τρόπος με τον οποίο τα διάφορα περιβάλλοντα κλήσης να μεταβιβάζουν παραμέτρους στην main. Ένας τύπος δεδομένων διαθέσιμος σε όλα τα περιβάλλοντα είναι η αλφαριθμητική σειρά. Με τη μορφή αλφαριθμητικής σειράς μπορεί να αναπαρασταθούν δεδομένα οποιουδήποτε τύπου. Για παράδειγμα, μια πραγματική τιμή μπορεί να αναπαρασταθεί ως αλφαριθμητική σειρά. Η τιμή 2.53 ας πούμε, μπορεί να αναπαρασταθεί ως “2.53”. Αυτός είναι ο λόγος που η main λαμβάνει ως παράμετρο έναν πίνακα από Strings. Μέσα από την παράμετρο αυτή, τα περιβάλλοντα κλήσης μπορούν να περάσουν τις παραμέτρους που απαιτούνται. Από τη στιγμή που η main θα έχει τις τιμές των παραμέτρων με τη μορφή αλφαριθμητικών τιμών, έχει τη δυνατότητα να τις μετατρέψει στους κατάλληλους τύπους και να εργαστεί με αυτές χωρίς πρόβλημα.

Ας πάρουμε όμως τα πράγματα από την αρχή. Ας θυμηθούμε τη διεπαφή της main.

```
public static void main(String[] args)
```

Καταρχάς, στην τυπική περίπτωση, η main δηλώνεται ως public. Αυτό είναι λογικό, διαφορετικά θα ήταν αδύνατο να κληθεί από τρίτους. Βέβαια ο μεταγλωττιστής δεν θα διαμαρτυρηθεί αν η main δηλωθεί με πρόσβαση πακέτου. Ενδεχομένως μάλιστα σε κάποιες ειδικές περιπτώσεις, όπου η main καλείται μέσα από Java, μια τέτοια δήλωση να είναι χρήσιμη. Ούτε και για τους υπόλοιπους προσδιοριστές προσπέλασης διαμαρτύρεται ο μεταγλωττιστής. Ωστόσο, σε όλες τις περιπτώσεις που η main θα τρέξει απευθείας από την εικονική μηχανή και όχι μέσα από κάποιον κώδικα Java, θα πρέπει να χρησιμοποιηθεί ο προσδιοριστής προσπέλασης public. Αν παρόλα αυτά επιχειρήσουμε να τρέξουμε μια εφαρμογή με μη δημόσια main, η εικονική μηχανή θα αποκριθεί με κατάλληλο μήνυμα στο οποίο θα αναφέρει ότι δεν βρίσκει main.

Επιπλέον, η main είναι δηλωμένη ως static. Πρόκειται για απαραίτητη δήλωση. Στην περίπτωση που θα δηλωθεί ως μη στατική συνάρτηση, η εικονική μηχανή θα αποκριθεί και πάλι πως δεν βρίσκει main. Η main ως κύρια είσοδος στην εφαρμογή πρέπει απαραίτητα να δηλωθεί ως στατική συνάρτηση, καθώς κλήση των μη στατικών συναρτήσεων μπορεί να πραγματοποιηθεί μόνο μέσα από εφαρμογή Java και άρα έπεται της εκκίνησης της εφαρμογής.

Επίσης, η main πρέπει να είναι τύπου void. Για αυτήν την απαίτηση ίσως να μην υπάρχει αναγκαιότητα. Θα μπορούσε να επιστρέφει int αντί για void όπως η main της C. Στην Java όμως έγινε αυτή η σχεδιαστική επιλογή. Η εικονική μηχανή ψάχνει μια main τύπου void.

Ας δούμε όμως μια πρότυπη main με παραμέτρους.

```
public class MainParameters {

    private static void syntax() {
        System.out.println("MainParameters int String");
        System.exit(1);
    }

    public static void main(String[] prms) {
        if (prms.length != 2) {
            syntax();
        }
        int firstArgument = Integer.parseInt(prms[0]);
        System.out.println("first Argument is " + firstArgument
                + "\nSecond Argument is " + prms[1]);
    }
```

```
}
```

Κώδικας 7.10 *Παράδειγμα main με παραμέτρους*

Στον κώδικα 7.10 παρουσιάζουμε την κλάση MainParameters η οποία περιέχει main με παραμέτρους. Βέβαια όλες οι main που είναι κύριες είσοδοι στο πρόγραμμα διαθέτουν τυπική παράμετρο τύπου String[]. Όσες είδαμε όμως μέχρι εδώ δεν χρησιμοποιούν την παράμετρο. Αντίθετα, η MainParameters τη χρησιμοποιεί.

Πιο συγκεκριμένα, η main εδώ χρειάζεται δύο παραμέτρους. Η πρώτη πρέπει να είναι τύπου int και η δεύτερη String. Οι τιμές των δύο παραμέτρων περνάνε στην main ως αλφαριθμητικές σειρές διαμέσου του πίνακα prms. Ας σημειωθεί εδώ πως το αναγνωριστικό prms δεν είναι παρά το αναγνωριστικό μιας παραμέτρου. Μπορεί ο προγραμματιστής που υλοποιεί την main να επιλέξει όποιο αναγνωριστικό κρίνει κατάλληλο. Συνήθως για την παράμετρο της main χρησιμοποιείται το αναγνωριστικό args.

Μέσα στον πίνακα prms λοιπόν αναμένει η main του κώδικα 7.10, δύο παραμέτρους. Η πρώτη που θα τοποθετηθεί στη θέση 0 του πίνακα θα αναπαριστά μια ακέραια τιμή και η δεύτερη που θα τοποθετηθεί στη θέση 1 θα είναι μια αλφαριθμητική σειρά.

Το πρώτο πράγμα που θα πρέπει να ελέγξουμε στην main είναι αν όντως κλήθηκε με σωστές παραμέτρους. Πράγματι, στην πρώτη γραμμή της main ελέγχουμε αν η εφαρμογή κλήθηκε με 2 παραμέτρους. Αν όχι, καλούμε τη συνάρτηση syntax.

Η syntax βγάζει ένα μήνυμα που ενημερώνει τον χρήστη πως η εφαρμογή μας καλείται με δύο παραμέτρους, μία τύπου int και μια τύπου String. Στη συνέχεια καλεί την System.exit με παράμετρο 1. Η System.exit προκαλεί άμεσο τερματισμό της εικονικής μηχανής και άρα και της εφαρμογής που την καλεί. Επομένως αν κληθεί η Syntax βγαίνει ένα μήνυμα στον χρήστη που υποδεικνύει τον ορθό τρόπο κλήσης και η εφαρμογή τερματίζεται καθώς χωρίς τις κατάλληλες παραμέτρους αδυνατεί να προχωρήσει.

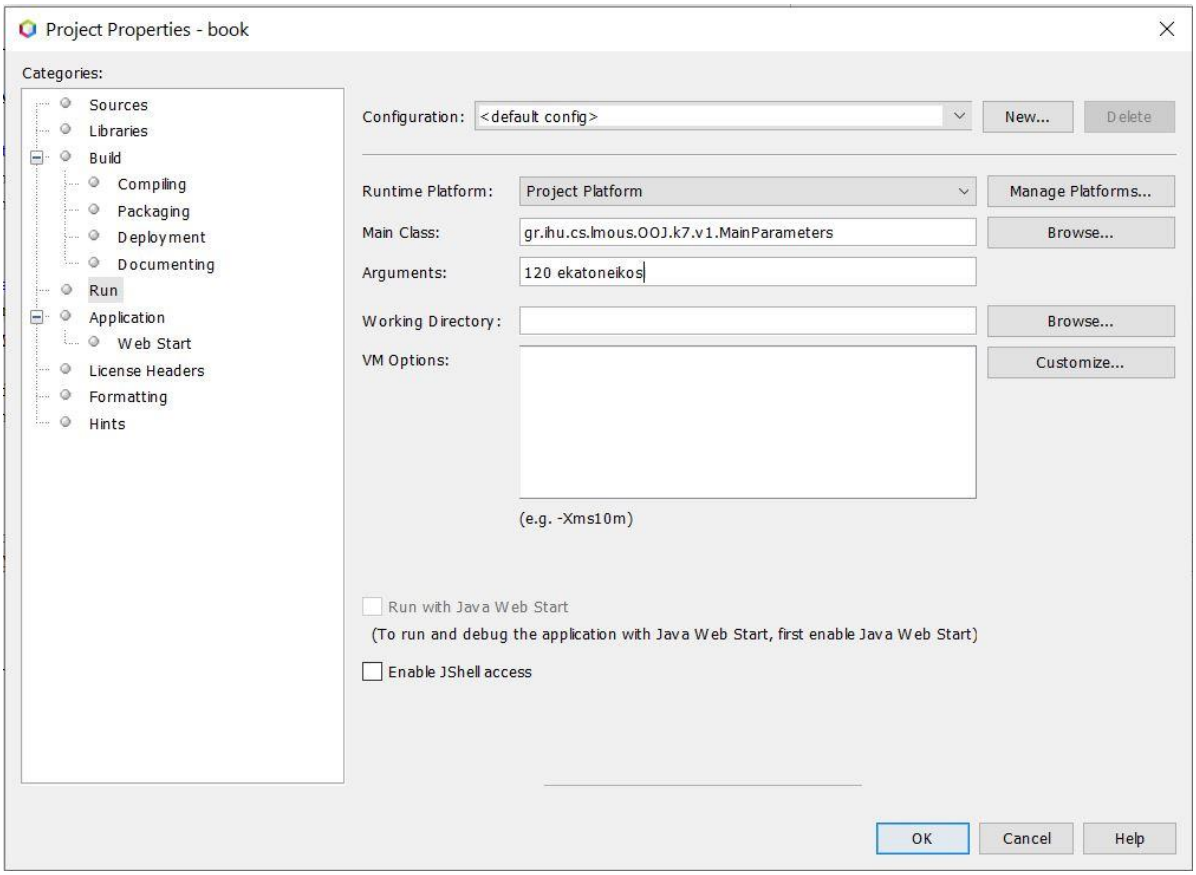
Αν δεν κληθεί η syntax, τότε η εφαρμογή επιχειρεί να μετατρέψει την πρώτη παράμετρο, δηλαδή αυτή που βρίσκεται στη θέση μηδέν του prms σε int. Έτσι όπως είναι ο κώδικας 7.10, αν η πρώτη παράμετρος δεν είναι μετατρέψιμη σε int, το πρόγραμμα πέφτει παράγοντας κατάλληλο μήνυμα. Αν η μετατροπή των παραμέτρων γίνει με επιτυχία, τότε η main έχει τις τιμές των παραμέτρων σε τοπικές μεταβλητές και μπορεί να προχωρήσει απρόσκοπτα στην ενδεδειγμένη επεξεργασία. Στον κώδικα 7.10 η πρώτη παράμετρος μετατρέπεται σε int με τη βοήθεια της Integer.parseInt και η τιμή της αποθηκεύεται στην τοπική μεταβλητή firstArgument. Η δεύτερη παράμετρος είναι τύπου String και δεν απαιτείται καμιά μετατροπή. Μπορεί να χρησιμοποιηθεί η prms[1].

Να επανέλθουμε λίγο στην System.exit και στην τιμή 1 που της περάσαμε ως πραγματική παράμετρο. Όπως έχουμε εξηγήσει, οι συναρτήσεις διαθέτουν μια τιμή επιστροφής που σκοπό έχει να ενημερώνει τον κώδικα που κάλεσε τη συνάρτηση με τα αποτελέσματα της κλήσης. Η main όμως είναι εξ ορισμού void. Η παράμετρος της System.exit αποτελεί έναν τρόπο για να ενημερώνουμε αυτόν που καλεί την εφαρμογή για το αποτέλεσμά της. Σε περίπτωση που η εφαρμογή έτρεξε ομαλά, η τιμή της παραμέτρου πρέπει να είναι μηδέν. Οποιαδήποτε άλλη τιμή μπορεί να χρησιμοποιηθεί για να επικοινωνήσει μη ομαλό τερματισμό της εφαρμογής. Από εκεί και μετά, οι πιθανές τιμές είναι θέμα σύμβασης μεταξύ της εφαρμογής και του περιβάλλοντος που την καλεί. Στα Windows για παράδειγμα, η τιμή της παραμέτρου της System.exit ενημερώνει τη μεταβλητή συστήματος errorlevel. Όλα τα scripts των Windows που καλούν κάποια εφαρμογή Java, μπορούν να ελέγχουν την errorlevel και να λαμβάνουν πληροφορίες σχετικά με τον τερματισμό της εφαρμογής.

Ας δούμε όμως δύο παραδείγματα κλήσης της MainParameters. Πιο συγκεκριμένα, θα δούμε πώς καλείται η MainParameters από το NetBeans και από το powershell των Windows.

7.3.4.1 Κλήση από το NetBeans

Στο τμήμα Projects του NetBeans, μεταβείτε στο project στο οποίο έχετε ορίσει την MainParameters και πατήστε δεξί κλικ. Θα ανοίξει ένα πτυσσόμενο μενού. Επιλέξτε Properties. Θα ανοίξει η φόρμα Project Properties.



Εικόνα 7.1 *Καθορισμός παραμέτρων της main στο NetBeans*

Στην αριστερή πλευρά της φόρμας Project Properties είναι τοποθετημένη μία λίστα κάτω από την επικεφαλίδα Categories. Στη λίστα Categories, επιλέξτε Run όπως φαίνεται στην εικόνα 7.1. Στη δεξιά πλευρά της φόρμας βρείτε τα πεδία Main Class και Arguments και ενημερώστε τα κατάλληλα. Στο πεδίο Main Class εισάγετε την κλάση MainParameters με τον προσδιοριστή πακέτου όπως φαίνεται στην εικόνα 7.1. Ο προσδιοριστής πακέτου έχει το όνομα του πακέτου στο οποίο ανήκει η κλάση. Μπορείτε να χρησιμοποιήσετε το button Browse για διευκόλυνσή σας. Στο πεδίο Arguments, τοποθετήστε τις παραμέτρους που απαιτεί η εφαρμογή διαχωρισμένες με ένα διάστημα. Πατήστε το button OK ώστε να αποθηκευτούν οι νέες ιδιότητες.

Μεταβείτε στο project που έχετε ορίσει την κλάση, πατήστε δεξί κλικ και από το πτυσσόμενο μενού, επιλέξτε Run. Στο παράθυρο Output του NetBeans θα δείτε να εμφανίζεται το αποτέλεσμα κλήσης της MainParameters.

7.3.4.2 Κλήση από το powershell

Μεταβείτε στο Project Properties. Στο παράθυρο Categories, επιλέξτε Sources. Επιλέξτε τη διαδρομή στην οποία είναι αποθηκευμένο το project σας από το πεδίο Project Folder. Χρησιμοποιήστε Ctrl+c για να αντιγράψετε τη διαδρομή του project στο πρόχειρο των Windows (clipboard).

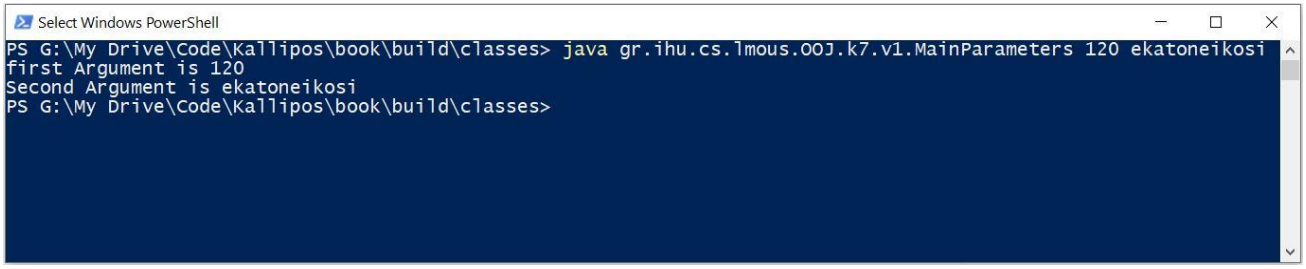
Ανοίξτε τον file explorer των Windows. Στη γραμμή εντολών δώστε Ctrl+V και Enter. Θα μεταβείτε έτσι στον φάκελο του project. Εκεί θα δείτε μια σειρά από φακέλους. Ο φάκελος build περιέχει φάκελο classes. Στον φάκελο classes είναι αποθηκευμένος ο bytecode της εφαρμογής σας οργανωμένος σε υποφακέλους ανάλογα με το πακέτο στο οποίο ανήκει η κάθε κλάση.

Μεταβείτε στον φάκελο classes. Ενώ βρίσκεστε στον φάκελο classes, πηγαίνετε στο μενού file του file explorer και κάντε κλικ στο Open Windows PowerShell. Θα ανοίξει το PowerShell με φάκελο εργασίας τον φάκελο classes.

Εκεί πληκτρολογήστε

```
Java packagePath.MainParameters first-argument second-argument
```

Στη θέση packagePath βάλτε το όνομα του πακέτου στο οποίο ανήκει η MainParameters. Η εικόνα 7.2 παρουσιάζει μια κλήση της MainParameter όταν αυτή έχει δηλωθεί να ανήκει στο πακέτο gr.ihu.cs.lmous.OOJ.k7.v1.



Εικόνα 7.2 *Κλήση εφαρμογής Java από το PowerShell των Windows.*

Όπως φαίνεται στην εικόνα 7.2, σε αυτό το παράδειγμα, η MainParameters καλείται με πρώτη παράμετρο το 112 και δεύτερη παράμετρο τη σειρά ekatondwdeka. Η λέξη Java στην αρχή της κλήσης ενεργοποιεί την JVM η οποία στην ουσία τρέχει την κλάση MainParameters.

7.4 Στατικές μεταβλητές

Η Java μας δίνει τη δυνατότητα να ορίσουμε στατικές μεταβλητές. Οι στατικές μεταβλητές ορίζονται στο επίπεδο της κλάσης και προσπελαύνονται μέσα από οποιαδήποτε συνάρτηση της κλάσης. Δεν υπάρχει δυνατότητα ορισμού στατικών μεταβλητών που είναι τοπικές μεταβλητές συναρτήσεων. Η πλήρης έννοια των στατικών μεταβλητών μπορεί να αναπτυχθεί μόνο στο πλαίσιο του Αντικειμενοστρεφούς μοντέλου όπου θα συζητηθούν και οι μη στατικές μεταβλητές και θα γίνει διαφοροποίηση μεταξύ στατικών και μη στατικών μεταβλητών.

Για την ώρα θα δούμε τη χρησιμότητα των στατικών μεταβλητών μέσα από ένα παράδειγμα αξιοποίησής των. Ας υποθέσουμε πως στην εφαρμογή που αναπτύσσουμε χρειαζόμαστε σε μορφή αλφαριθμητικής σειράς το άθροισμα ή και το γινόμενο πραγματικών αριθμών αποθηκευμένων σε κατάλληλο πίνακα. Σε αυτήν την περίπτωση είναι λογικό να υλοποιήσουμε δύο συναρτήσεις, μία που υπολογίζει και επιστρέφει το άθροισμα και μια για το γινόμενο. Θέλουμε όμως οι δύο συναρτήσεις να διαμορφώνουν τα αποτελέσματά τους με κοινό μοτίβο. Σκοπός μας είναι να εμφανίσουμε άθροισμα και γινόμενο και δεν επιθυμούμε να εμφανίζονται με διαφορετικό πλήθος δεκαδικών. Επομένως χρειαζόμαστε έναν τρόπο διαμόρφωσης του αποτελέσματος κοινό για τις δύο συναρτήσεις. Εδώ μας διευκολύνουν οι στατικές μεταβλητές της κλάσης:

```
import java.text.DecimalFormat;

public class StaticVars {

    public static String format; // "###.##"

    public static String sum(double[] in) {
        double sum = 0;
        for (double current : in) {
            sum += current;
        }
        DecimalFormat f = new DecimalFormat(format);
        return f.format(sum);
    }

    public static String product(double[] in) {
        double product = 1;
        for (double current : in) {
            product *= current;
        }
        DecimalFormat f = new DecimalFormat(format);
```

```
        return f.format(product);
    }

    public static void main(String[] args) {
        format = "###.##";
        double[] d = {5.6, 7.95, 3.23, 45, 96};
        System.out.println(sum(d));
        System.out.println(product(d));
    }
}
```

Κώδικας 7.11 *Χρήση στατικής μεταβλητής*

Στον κώδικα 7.11 δίνεται ο ορισμός της κλάσης StaticVars. Η κλάση περιλαμβάνει ορισμό των συναρτήσεων sum και product. Και οι δύο λαμβάνουν ως παράμετρο έναν πίνακα πραγματικών. Η sum υπολογίζει το άθροισμα των τιμών της παραμέτρου της και η product το γινόμενο. Και οι δύο υπολογίζουν το αποτέλεσμά τους και το επιστρέφουν ως String. Στην κλάση ορίζεται και η στατική μεταβλητή format. Σκοπός της είναι να ελέγχει τη διαμόρφωση κατά την εμφάνιση των πραγματικών μεταβλητών. Ως στατική μεταβλητή της κλάσης, η format μπορεί να προσπελαστεί από όλες τις συναρτήσεις της κλάσης. Στην main, πριν καλέσουμε τις μεθόδους, φροντίζουμε να ενημερώσουμε κατάλληλα την format. Στη συνέχεια, οι δύο μέθοδοι διαβάζουν την format και επιστρέφουν το αποτέλεσμά τους διαμορφωμένο σύμφωνα με το μοτίβο που αποθηκεύσαμε στην format.

Κατά μία έννοια, μια στατική μεταβλητή αποτελεί μία κοινή μνήμη για όλη την κλάση.

Ας υποθέσουμε τώρα πως χρειαζόμαστε την ίδια λειτουργικότητα, δηλαδή να εμφανίζουμε το άθροισμα και το γινόμενο των στοιχείων πινάκων πραγματικών και σε μία άλλη εφαρμογή. Έστω όμως πως στην άλλη εφαρμογή, χρειαζόμαστε διαφορετική διαμόρφωση, π.χ. θέλουμε τρία δεκαδικά ψηφία αντί για δύο. Με δεδομένο ότι οι συναρτήσεις sum και product αλλά και η σειρά format είναι δημόσια μέλη της κλάσης, η λύση είναι απλή και παρατίθεται στον κώδικα 7.12.

```
public class AnotherApp {

    public static void main(String[] args) {
        StaticVars.format = "###.###";
        double[] d = {5.675, 7.915, 3.213, 45, 0.96};
        System.out.println(StaticVars.sum(d));
        System.out.println(StaticVars.product(d));
    }
}
```

Κώδικας 7.12 *Χρήση στατικής μεταβλητής από άλλη κλάση*

Στον κώδικα 7.12, η main της κλάσης AnotherApp δίνει τιμή στη στατική μεταβλητή format της StaticVars. Στη συνέχεια καλεί τις sum και product της StaticVars προκειμένου να εμφανίσει το άθροισμα και το γινόμενο των στοιχείων του πίνακα d.

Όπως θα αντιληφθήκατε ήδη, προκειμένου να προσπελάσουμε μια στατική συνάρτηση ή μια στατική μεταβλητή από κλάση διαφορετική από αυτήν που ορίζονται πρέπει να προηγηθεί του ονόματός τους, το όνομα της κλάσης στην οποία ορίζονται. Σε αυτήν τη χρήση, το όνομα της κλάσης θεωρείται ένα προσδιοριστικό ονόματος (name specifier).

Αν έχουμε σύγκρουση μεταξύ αναγνωριστικού στατικής μεταβλητής μιας κλάσης με τοπική μεταβλητή μέσα σε συνάρτηση της ίδιας κλάσης, τότε θα πρέπει να χρησιμοποιήσουμε τον προσδιοριστή ονόματος κατά την αναφορά στη στατική μεταβλητή.

7.5 Απροσδόκητα λάθη

Ας υποθέσουμε πως θέλουμε να αναπτύξουμε μια συνάρτηση που υπολογίζει το παραγοντικό μιας ακέραιας τιμής. Το πρώτο βήμα βέβαια θα είναι να βεβαιωθούμε πως γνωρίζουμε τον σωστό ορισμό του παραγοντικού.

Για τη διευκόλυνση της συζήτησής μας παραθέτουμε τον ορισμό.

Το παραγοντικό ενός μη αρνητικού ακέραιου n, συμβολίζεται με n! και είναι ίσο με 1 αν n=0 ή n=1, ενώ αν n>1, είναι το γινόμενο όλων των θετικών ακέραιων που είναι μικρότεροι ή ίσοι με το n.

Με δεδομένο τον ορισμό, η ανάπτυξη της συνάρτησης είναι σχετικά απλή. Ο κώδικας 7.13 μας δίνει τη συνάρτηση του παραγοντικού.

```
public class UnexpectedErrors {

    static long factorial(long n) { //Κώδικας 7.13
        if (n == 0 || n == 1) {
            return 1;
        }
        long rVal = 1;
        for (long i = 2; i <= n; i++) {
            rVal *= i;
        }
        return rVal;
    }
}
```

Κώδικας 7.13 *Υπολογισμός παραγοντικού*

Η συνάρτηση factorial είναι τύπου long καθώς το παραγοντικό μεγαλώνει γρήγορα σε σχέση με το όρισμά του. Για παράδειγμα, 3! = 6, 6! = 720 και 9! = 362880. Στην αρχή της συνάρτηση factorial, ελέγχεται το n καθώς αν είναι 0 ή 1 δεν απαιτούνται περαιτέρω υπολογισμοί μια και για αυτές τις τιμές του n, το n! είναι ίσο με 1. Στη συνέχεια αρχικοποιείται η rVal στην τιμή 1 και πολλαπλασιάζεται διαδοχικά με όλους τους ακέραιους που είναι μεγαλύτεροι ή ίσοι του 2 και μικρότεροι ή ίσοι του n. Με την έξοδο από τον βρόχο for, η rVal αναπαριστά την απαιτούμενη τιμή την οποία η συνάρτηση επιστρέφει.

Αν τρέξετε την factorial, θα διαπιστώσετε ότι επιστρέφει τη σωστή τιμή για όλες τις μη αρνητικές τιμές της παραμέτρου n.

Τι θα συμβεί όμως αν η factorial κληθεί με πραγματική παράμετρο έναν αρνητικό ακέραιο; Καταρχάς, δεν θα επιστρέψει από την αρχική if. Στη συνέχεια θα αρχικοποιήσει την rVal στο 1. Στη συνέχεια δεν θα μπει στον βρόχο for, καθώς το θετικό i δεν είναι μικρότερο ή ίσο από οποιονδήποτε αρνητικό αριθμό και τέλος θα επιστρέψει το rVal του οποίου η τιμή παραμένει 1. Με λίγα λόγια, η factorial του κώδικα 7.13 επιστρέφει την τιμή 1 ως αποτέλεσμα υπολογισμού του παραγοντικού οποιουδήποτε αρνητικού ακέραιου.

Αυτό όμως είναι λάθος, μιας και δεν ορίζεται παραγοντικό για αρνητικούς ακέραιους. Πρόκειται μάλιστα για σοβαρό λάθος με απρόβλεπτες συνέπειες κατά την εκτέλεση της εφαρμογής που το περιέχει.

Μπορεί να αναρωτιέστε γιατί να κληθεί η factorial με αρνητικό ακέραιο. Ενδεχομένως, η factorial να καλείται με όρισμα κάποια μεταβλητή η οποία να προκύπτει από κάποιους υπολογισμούς. Ενδεχομένως, το λάθος να ξεκινά από εκεί. Γενικά, στις ρεαλιστικές εφαρμογές όπου εμπλέκονται πάρα πολλές μεταβλητές και πάρα πολλοί και σύνθετοι υπολογισμοί, τέτοιου είδους λάθη είναι σχεδόν αδύνατο να αποκλειστούν. Εκείνο όμως που μπορεί να γίνει είναι να εντοπιστούν εγκαίρως και να διορθωθούν. Σε αυτό μας βοηθάνε οι εξαιρέσεις (Exceptions) της Java.

Η διαχείριση των απροσδόκητων λαθών είναι μεγάλο κομμάτι κάθε γλώσσας προγραμματισμού και της Java. Αντίστοιχα, πολλές είναι και οι κλάσεις που αφορούν τη διαχείριση των λαθών. Υπάρχουν διαχειρίσιμα και μη διαχειρίσιμα λάθη. Λεπτομέρειες για τη διαχείριση των λαθών συζητάμε στην ενότητα 16. Για την ώρα μας αρκεί μια προσέγγιση που αν και απλή μας βοηθάει να προστατευτούμε από τα απροσδόκητα λάθη στον κώδικά μας. Η προσέγγιση λοιπόν που προτείνουμε εδώ βασίζεται στην παραγωγή μιας εξαίρεσης τύπου RuntimeException. Με άλλα λόγια, στην αρχή της συνάρτησης factorial θα ελέγξουμε το όρισμα και αν βρεθεί αρνητικό θα παράγουμε μια εξαίρεση τύπου RuntimeException. Η παραγωγή της εξαίρεσης θα προκαλέσει τερματισμό του προγράμματος με εμφάνιση κατάλληλου μηνύματος. Έτσι θα βοηθηθούμε να εντοπίσουμε το πρόβλημα και να το διορθώσουμε.

Καλό είναι αυτός που καλεί τη συνάρτησή μας να γνωρίζει πως η συνάρτηση ενδέχεται να παράγει εξαίρεση. Έτσι, η Java υποστηρίζει τη δήλωση των εξαιρέσεων που ενδέχεται να παράγει μια συνάρτηση στην επικεφαλίδα της. Ο κώδικας 7.14 παρουσιάζει την factorial με ενσωματωμένες τις παρατηρήσεις που κάναμε σε αυτήν την ενότητα.

```
public class UnexpectedErrors {
```

```
    static long factorial(long n) throws RuntimeException { //Κώδικας 7.14
        if (n < 0) {
            throw new RuntimeException();
        }
        if (n == 0 || n == 1) {
            return 1;
        }
        long rVal = 1;
        for (long i = 2; i <= n; i++) {
            rVal *= i;
        }
        return rVal;
    }
}
```

Κώδικας 7.14 *Η factorial με έλεγχο της παραμέτρου*

Όπως φαίνεται στον κώδικα 7.14, η παραγωγή της εξαίρεσης γίνεται με τη λέξη-κλειδί throw. Με τη βοήθεια της λέξης-κλειδί throws δηλώνεται προαιρετικά στην επικεφαλίδα της συνάρτησης το γεγονός ότι η συνάρτηση ενδέχεται να παράγει εξαίρεση τύπου RuntimeException.

Τρέξτε τη νέα έκδοση της factorial με αρνητικό όρισμα και μελετήστε το αποτέλεσμα.

7.6 Χρήσιμες συναρτήσεις

Σε αυτήν την ενότητα παρουσιάζουμε συναρτήσεις ενσωματωμένες στο βασικό πλαίσιο της Java που συχνά μας είναι χρήσιμες. Όπως θα έχετε ήδη αντιληφθεί, κάθε συνάρτηση στην Java ανήκει και σε μία κλάση. Επιπρόσθετα λοιπόν κάποιων συναρτήσεων της βιβλιοθήκης της Java που έχουμε ήδη δει, στην ενότητα αυτή θα δούμε κάποιες συναρτήσεις που ανήκουν στις κλάσεις Math και String.

7.6.1 Η κλάση Math

Η κλάση Math περιλαμβάνει μια σειρά από στατικές συναρτήσεις και στατικές σταθερές μεταβλητές που μας επιτρέπουν να κάνουμε βασικούς μαθηματικούς υπολογισμούς. Παρουσιάζουμε εδώ μέρος των συναρτήσεων και σταθερών μεταβλητών της Math. Αυτές που θεωρούμε πως είναι πιο αναγκαίες στα πλαίσια αυτού του εγχειριδίου. Για κάθε μια συνάρτηση παρουσιάζουμε πρώτα τη διεπαφή της ακολουθούμενη από μια μικρή περιγραφή και στη συνέχεια επιδεικνύουμε τον τρόπο κλήσης της.
Η σταθερά E αναπαριστά το e, τη βάση των φυσικών λογάριθμων. Η διεπαφή της έχει ως εξής:

```
public static final double E
```

Η σταθερά PI αναπαριστά το π, την αναλογία της περιφέρειας ενός κύκλου προς τη διάμετρο του κύκλου. Η διεπαφή της είναι:

```
public static final double PI
```

Η συνάρτηση toRadians μετατρέπει τις μοίρες σε ακτίνια.

```
public static double toRadians(double angdeg)
```

Η παράμετρος angdeg αναπαριστά τις μοίρες μιας γωνίας. Η συνάρτηση επιστρέφει τα ακτίνια της ίδιας γωνίας.

Η συνάρτηση toDegrees μετατρέπει τα ακτίνια σε μοίρες.

```
public static double toDegrees(double angrad)
```

Η παράμετρος angrad αναπαριστά τα ακτίνια μιας γωνίας. Η συνάρτηση επιστρέφει τις μοίρες της ίδιας γωνίας.

Η συνάρτηση sin επιστρέφει το ημίτονο μιας γωνίας.

```
public static double sin(double a)
```

Η παράμετρος a αναπαριστά τα ακτίνια μιας γωνίας. Αν επομένως θέλετε να υπολογίσετε το ημίτονο μιας γωνίας 30 μοιρών, δεν έχετε παρά να καλέσετε

```
Math.sin(Math.toRadians(30))
```

Η συνάρτηση cos επιστρέφει το συνημίτονο μιας γωνίας.

```
public static double cos(double a)
```

Η παράμετρος a αναπαριστά τα ακτίνια μιας γωνίας. Αν επομένως θέλετε να υπολογίσετε το συνημίτονο των 30 μοιρών καλέστε

```
Math.cos(Math.toRadians(30))
```

Η συνάρτηση tan επιστρέφει την εφαπτομένη γωνίας.

```
public static double tan(double a)
```

Η παράμετρος a αναπαριστά τα ακτίνια μιας γωνίας. Αν επομένως θέλετε να υπολογίσετε την εφαπτομένη των 30 μοιρών καλέστε

```
Math.tan(Math.toRadians(30))
```

Η συνάρτηση log επιστρέφει τον φυσικό λογάριθμο της παραμέτρου της.

```
public static double log(double a)
```

Για να υπολογίσετε τον φυσικό λογάριθμο του 90, καλέστε

```
Math.log(90)
```

Η συνάρτηση log10 επιστρέφει τον λογάριθμο με βάση το 10.

```
public static double log10(double a)
```

Για να υπολογίσετε τον λογάριθμο με βάση 10 του 90, καλέστε

```
Math.log10(90)
```

Η συνάρτηση pow επιστρέφει τη δύναμη ενός αριθμού υψωμένου σε έναν εκθέτη.

```
public static double pow(double a,  double b)
```

Η παράμετρος a αναπαριστά τη βάση και η b τον εκθέτη. Για να υπολογίσετε την παράσταση 5.24 καλέστε

Math.pow(5.2, 4)

Η συνάρτηση sqrt επιστρέφει την τετραγωνική ρίζα του ορίσματός της.